



```
a=[1,2,3,4,5]
print(a)
```

```
[1, 2, 3, 4, 5]
```

```
import numpy as np
arr=np.array([1,2,3,4,5])
print(arr)
```

```
[1 2 3 4 5]
```

```
a=[1,2,3]
b=[4,5,6]
res=[]
for i in range(len(a)):
    res.append(a[i]+b[i])
print(res)
```

```
[5, 7, 9]
```

```
import numpy as np
a=np.array([1,2,3])
b=np.array([4,5,6])
print(a+b)
print(a-b)
print(a*b)
print(a**b)
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[ 1 32 729]
```



Array creation in numpy

```
import numpy as np
a=np.array([1,2,3,4,5]) #1d
b=np.zeros(5)
print(a)
print(b)
c=np.zeros((2,2))#2d
print(c)
d=np.ones(3) #1row 1d
print(d)
e=np.ones((2,3))
print(e)
g=np.ones((1,3,2)) #for 3d
print(g)
print(np.arange(1,10,2)) #work like range using stepvalue
print(np.linspace(0,20,5)) #split in decimal
print(np.linspace(0,20,6)) #split 6 values by 4
print(np.linspace(0,20,3))
print(np.random.rand(5)) #generate some random values from 0 to 1
print(np.random.randint(1,100,7)) #for integer values 1-start,100-end,how many int values we want is 7
print(np.random.randn(5)) #for decimal values both negative and positive values
```

```

import numpy as np #dimensional arrays
arr=np.array([1,2,3,4,5])
ar=np.array([[1,2,3],[4,5,6]])
a=np.array([[[1,2],[3,4],[5,6]]])
print(arr)
print(ar)
print(a)
print(ar.shape) #find shape for ar
print(arr.shape)
print(a.shape)
print(arr.ndim) #rows in arr
print(ar.ndim) #for ar it is 2d show 2 row
print(a.ndim) #for multidimensional there is no shape
print(arr.size) #total no of val count
print(ar.size)
print(a.size)
print(a.dtype) #shows datatype
s=np.array([[
    [1,2],[9,7]],
    [[3,4],[4,5]],
    [[6,7],[9,8]]])
print(s.shape)

```

```

[1 2 3 4 5]
[[1 2 3]
 [4 5 6]]
[[[1 2]
  [3 4]
  [5 6]]]
(2, 3)
(5,)
(1, 3, 2)
1
2
3

```

```

[1 2 3 4 5]
[0. 0. 0. 0. 0.]
[[0. 0.]
 [0. 0.]]
[1. 1. 1.]
[[1. 1. 1.]
 [1. 1. 1.]]
[[[1. 1.]
   [1. 1.]
   [1. 1.]]]
[1 3 5 7 9]
[ 0.  5. 10. 15. 20.]
[ 0.  4.  8. 12. 16. 20.]
[ 0. 10. 20.]
[0.53453921 0.24746979 0.61357571 0.27059718 0.24163845]
[38 98 20 54 66 16 64]
[ 0.01260212 -1.94575814  0.11624159  0.60166749  0.09325555]

```

numpy indexing and slicing

```

import numpy as np
a=np.array([10,20,30,43,50,60,67])
#1d array indexing
print(a[2])
print(a[-1]) #from last
print(a[::-2])
b=np.array([[1,2,3],
            [4,5,6],
            [7,8,9]])
#2d array indexing
print(b[1][1]) #(1,1)=5
#1d slicing
print(a[2:7]) #from index of 2-7 only start and stop values
print(a[:4])

```

```
print(a[:,2])
print(a[:,1]) #whole print
#2d slicing
print(b[1]) #row 1 print
print(b[:,1]) #1st column ,([:,1])shows error so b is must,comma for column
print(b[:,-1])
print(a[a>50]) #filter based on condition not true or false(boolean indexing)
print(a[a<50])
print(a[[0,4,3,6]]) #select multiple specify element using index list(Fancy indexing),multiple val return so multiple[]
print(a[a%2==0]) #print only even numbers
a[4]=1000 #add values in index 4
print(a)
```

```
30
67
[67 50 30 10]
5
[30 43 50 60 67]
[10 20 30 43]
[10 30 50 67]
[10 20 30 43 50 60 67]
[4 5 6]
[2 5 8]
[3 6 9]
[60 67]
[10 20 30 43]
[10 50 43 67]
[10 20 30 50 60]
[ 10  20  30  43 1000  60  67]
```

mathematical operation in numpy

```
] import numpy as np
a=np.array([1,2,3])
print(a+10)    #scalar broadcasting in 1d add for each value
b=np.array([[1,2,3],[4,5,6]]) #2d broadcasting
print(b+10)
print(b+a)
#always no of columns must be same
```

```
[11 12 13]
[[11 12 13]
 [14 15 16]]
[[2 4 6]
 [5 7 9]]
```

matrix operation

```
] import numpy as np
a=np.array([[1,2],
            [3,4]])
b=np.array([[4,3],
            [2,1]])
print(a*b)
print(np.dot(a,b)) #product and add,method1
print(a@b) #method2
print(np.matmul(a,b)) #method3
print(a.T) #transpose-row become col and col become row
```

```
[[4 6]
 [6 4]]
[[ 8  5]
 [20 13]]
```

```
[[ 8  5]
 [20 13]]
[[ 8  5]
 [20 13]]
[[1 3]
 [2 4]]
```

universal mathematical function

```
75]: import numpy as np
a=np.array([4,9,16])
print(np.sqrt(a)) #square numbers
b=np.array([-5,-6,-2,-8])
print(np.abs(b)) #absolute values neg to pos
print(np.log(a)) #log values
print(np.exp(a)) #pow valuse
```

```
[2.  3.  4.]
[5  6  2  8]
[1.38629436  2.19722458  2.77258872]
[5.45981500e+01  8.10308393e+03  8.88611052e+06]
```

```
78]: ## round function
arr=np.array([1.234,5.674])
print(np.round(arr,2)) #after decimal only 2 val print
print(np.floor(arr)) #roundoff, Low val
print(np.ceil(arr)) #high val
```

```
[1.23  5.67]
[1.  5.]
[2.  6.]
```



```

#statistical function
import numpy as np
a=np.array([10,2,3,40,50])
print(np.mean(a))
print(np.median(a)) #after sorting middle value return so 10
print(np.std(a))
print(np.max(a))
print(np.min(a))
print(np.percentile(a,0)) #after sorting 25% is 3
print(np.percentile(a,25))
print(np.percentile(a,50))
print(np.percentile(a,75))
print(np.percentile(a,100))
print(np.var(a))
print(np.argmin(a)) #min val of index ,it doesn't consider 0
print(np.argmax(a)) #max val of index
b=np.array([1,2,3,4,5])
c=np.array([2,4,6,8,10])
print(np.corrcoef(b,c)) #returns pos or neg relationship.both increase pos ,if anyone dec it gives negative
d=np.array([1,2,3,4,5])
e=np.array([10,8,6,4,2])
print(np.corrcoef(d,e))
#Aggregate function

d=np.array([[1,3,4],
            [5,6,8]])
print(np.sum(d))
print(np.sum(d,axis=1)) #row axis=1
print(np.sum(d,axis=0)) #col axis=0

```

```

21.0
10.0
20.039960079800558
50

```



```
2
2.0
3.0
10.0
40.0
50.0
401.6
1
4
[[1. 1.]
 [1. 1.]]
[[ 1. -1.]
 [-1.  1.]]
27
[ 8 19]
[ 6  9 12]
```

Problems

```
[113]: import numpy as np
scores=np.array([[20,30,40],
                 [50,60,70],
                 [80,90,100]])

print(scores)
print(np.mean(scores))
print(np.max(scores,axis=1))

norm=(scores-np.min(scores))/(np.max(scores)-np.min(scores))
print(norm)
print(np.mean(scores,axis=0)) #col avg

[[ 20  30  40]
 [ 50  60  70]
 [ 80  90 100]]
```

```
60.0  
[ 40  70 100]  
[[0.    0.125 0.25 ]  
 [0.375 0.5   0.625]  
 [0.75  0.875 1.   ]]  
[50. 60. 70.]
```

```
9]: import numpy as np  
arr=np.array([10,20,30,40,50])  
s=arr[arr>25]  
print(s)  
total=sum(s)  
print(total)
```

```
[30 40 50]  
120
```

```
1]: import numpy as np  
a=np.array([[1],  
            [2],  
            [3]])  
b=np.array([[4],[5],[6]])  
print(a*b)
```

```
[[ 4]  
 [10]  
 [18]]
```

```
]:
```